Coding Challenge: Interactive Todo List (CRUD with Initial API Data)

Objective:

Build a Next.js application using the App Router and TypeScript that fetches a list of todos from a public API, uses 3 random entries to initially populate an interactive list, and allows the user to add, edit, and delete todos from this list.

Concepts Covered:

- Creating a Next.js project with TypeScript and App Router.
- Fetching data from an external API in a Server Component for initial data.
- Passing data from a Server Component to a Client Component.
- Creating and using a Client Component for interactivity.
- Client-side state management using `useState`.
- Implementing Create, Read, Update, and Delete (CRUD) operations on client-side state.
- Handling user input with forms.
- Rendering lists of data dynamically (`map`, `key`).
- Basic TypeScript usage for data types and code organization.
- Basic error handling for the initial API fetch.
- Conditional rendering based on data, error, or state.
- Applying basic CSS styling.

The API:

You will use the **JSONPlaceholder API**:
`https://jsonplaceholder.typicode.com/todos`

This API returns an array of todo objects. Each todo object has the following structure:

JSON

```JavaScript
[
  {
```

```
    "userId": 1,
    "id": 1,
    "title": "delectus aut autem",
    "completed": false
  },
  // ... more todo objects
]
```

Challenge Requirements:

1. Project Setup:

- Create a new Next.js project using `create-next-app` with the **App Router** and **TypeScript**.
- Include ESLint and Tailwind CSS (optional, but recommended for easier styling).

2. API Data Fetching (Server Component):

- Create a new page in your `app` directory (e.g., `app/todos/page.tsx`).
- Inside this **Server Component** page, fetch the *entire list* of todos from `https://jsonplaceholder.typicode.com/todos`.
- Implement basic error handling for this fetch. If it fails, the page should indicate that initial data could not be loaded.

3. Initial Data Prepopulation:

- If the API fetch is successful, select **3 random** todo objects from the fetched array. These 3 will be the initial state of your interactive todo list. Make sure each of these initial todos has a unique identifier (the `id` from the API is suitable).

4. TypeScript Types & Code Structure:

- Create a separate file (e.g., `lib/types.ts`) to define a TypeScript interface or type for a single todo object (`userId`, `id`, `title`, `completed`).
- Create a **Client Component** (e.g., `components/TodoList.tsx`) that will handle the display and interaction of the todo list. Remember to add the `'use client'` directive at the top of this file.
- The Server Component page (`app/todos/page.tsx`) will pass the array of 3 random initial todos as a prop to the `TodoList` Client Component.

5. Client-Side State Management & Display (Client Component):

- In the `TodoList` Client Component, use the `useState` hook to manage the current list of todos. Initialize this state with the array of 3 random todos passed via props.
- Render the current list of todos from the component's state. For each todo, display its `title` and whether it's `completed`. Include buttons or elements for Edit and Delete actions next to each todo.

6. CRUD Functionality (Client Component):

- **Add:**
  - Create a form (or input field and button) to add a new todo.
  - When the form is submitted or the button is clicked, add a *new* todo object to the component's state. Assign it a unique temporary ID (e.g., using `Date.now()`, a simple counter, or a library like `uuid`). Set `completed` to `false` and `userId` to a placeholder like `1`. The `title` comes from the input field.
- **Edit:**
  - Next to each displayed todo, provide a way to edit its `title`. This could be a button that reveals an input field, or making the title itself editable.
  - When a todo is edited and saved, update the corresponding todo object in the component's state.
- **Delete:**
  - Next to each displayed todo, provide a Delete button.
  - When the Delete button is clicked, remove the corresponding todo object from the component's state.

7. Basic Styling:

- Add basic styling to the page and the todo list.
- Style the list items, the form, and the buttons to be clearly distinguishable and easy to use.
- Ensure the error message (if the initial fetch fails) is clearly displayed.

8. Conditional Rendering:

- The Server Component should conditionally render either the `TodoList` Client Component (passing the initial data) or an error message based on the API fetch result.
- The Client Component should render the list of todos based on its internal state.

Step-by-Step Guide (High-Level):

1. Create the Next.js project with App Router and TypeScript.
2. Define the `Todo` interface in `lib/types.ts`.
3. Create the `app/todos/page.tsx` Server Component.
4. Inside `app/todos/page.tsx`:
   - Use a `try...catch` block to fetch data from `https://jsonplaceholder.typicode.com/todos`.
   - Check if the `response.ok` is true. If not, throw an error.
   - Parse the JSON response, asserting its type is `Todo[]`.
   - If successful, implement logic to select 3 random todos from the array. A simple way is to shuffle the array and take the first 3.
   - If an error occurs during the fetch or processing, set an error indicator (e.g., a state variable or by throwing an error that Next.js can handle with an `error.tsx` file, though a simple conditional render is fine for this level).
   - Import your `TodoList` Client Component.
   - In the `return` statement, conditionally render `<TodoList initialTodos={yourRandomTodos} />` if successful, or an error message (e.g., `<div>Failed to load initial todos.</div>`) if not.
5. Create the `components/TodoList.tsx` Client Component. Add `'use client'` at the top.
6. Define the props for `TodoList` to accept `initialTodos: Todo[]`.
7. Inside `TodoList.tsx`:
   - Import `useState`.

    - Initialize state: `const [todos, setTodos] = useState<Todo[]>(initialTodos);`.
    - Implement the JSX to display the list (`map` over `todos`), the Add form, and Edit/Delete buttons/logic for each item. You'll likely need additional state to manage which todo is currently being edited.
    - Write the `addTodo`, `editTodo`, and `deleteTodo` functions. These functions will take the necessary data (e.g., new title, todo ID) and use `setTodos` with a function updater (`setTodos(prevTodos => ...)`) to modify the array in state immutably (e.g., using `filter` for delete, `map` for edit, spread syntax `...` for add).
    - Tie the form submission and button clicks to these state-updating functions.
8. Add CSS styling (in `app/globals.css`, CSS Modules, or via Tailwind classes) to style the page, list, and form.
9. Run the development server (`npm run dev`).
10. Test the `/todos` page to ensure it loads with 3 random items, and that you can add, edit, and delete items in the list.

Bonus Challenges (for those who want to go further):

- Share your project using a public git repository (Gitlab, Github, etc).
- Implement a loading state using the App Router's loading.tsx convention or by converting the initial fetch to a client-side effect (less ideal for initial data, but good practice).
- Add validation to the Add/Edit form (e.g., ensure the title is not empty).
- Implement a "Toggle Complete" functionality for each todo.
- Add filtering (e.g., show all, show active, show completed).
- Persist the todo list changes using Server Actions or a dedicated API Route that interacts with a real database or a mock backend service (JSONPlaceholder does not support actual data persistence).
- Improve the UI/UX significantly (animations, better form handling).

Submission:

When you are ready, we will get on a screenshare where you will demo your app highlighting the following points:

- ☐ Does it work?
    - ☐ Can you access the app from a browser (locally)?
    - ☐ Can you create, delete, and edit todos from the browser?
- ☐ Where in the app handles the todo state?
- ☐ What did you have to learn to complete this project? Talk about your learning process.

- [ ] What layer was your favorite to work on? Ie data layer, presentation layer, styling, etc